C++ Templates and STL

Advanced Embedded Software Development

with **Dan Walkes**



University of Colorado **Boulder**

Learning objectives:

Introduce C++ Templates

Introduce the C++ Standard Template Library

Introduce boost

C++ Templates

```
int min(int a, int b)
{
  return a < b ? a: b;
}
```

```
double min(double a, double b)
{
  return a < b ? a : b;
}
```

- C/C++ Strong Typing means we need to write custom functions for different types
- Time consuming, tedious, error prone

https://blog.feabhas.com/2014/05/an-introduction-to-c-templates/

C++ Templates

```
int min(int a, int b)
{
  return a < b ? a: b;
}
```

```
double min(double a, double b)
{
  return a < b ? a : b;
}
```

```
#define min(a,b)((a) < (b)? (a): (b))
```

- C solution is to use a macro and the pre-processor. Drawbacks:
  - Parameters are not type-checked (in this example)
  - Parameters are not evaluated before pasting in macro

https://blog.feabhas.com/2014/05/an-introduction-to-c-templates/

C Macro Parameter Evaluation

```
#define min(a,b)((a) < (b)? (a): (b))
```

- A macro just pastes entire non-evaluated in, including any expression.
- Evaluation happens on each replacement
- Can result in unexpected and difficult to understand side-effects

Source code

```
int main()
{
  int i = 10;
  int j = 5;

  int m = min(++i, ++j)  // m = 6?
}
```

Compiled code

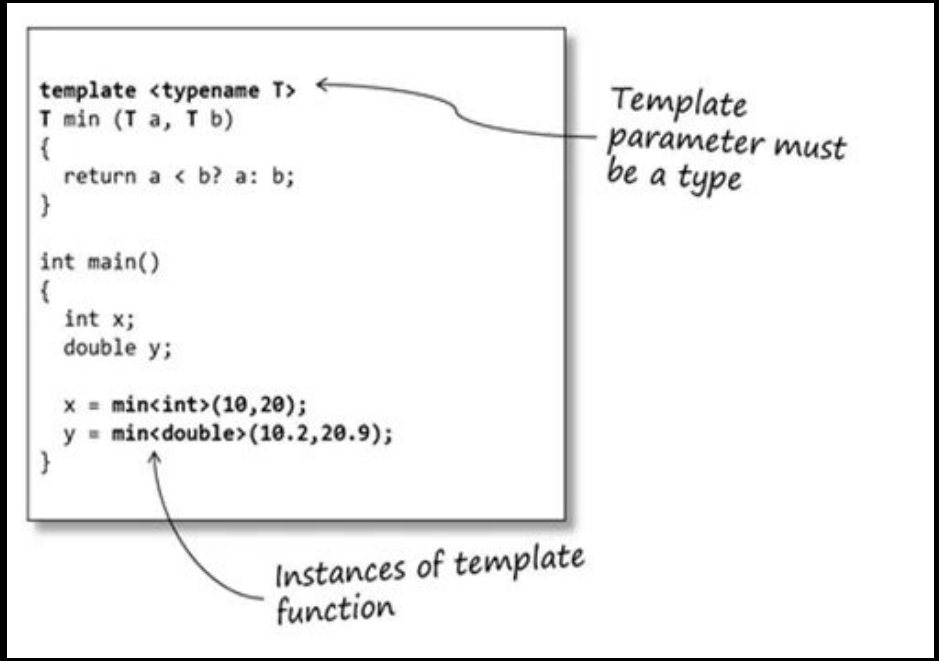
```
int main()
{
  int i = 10;
  int j = 5;

  int m = (++i) < (++j)? (++i) : (++j); // m = 7!
}
```

https://blog.feabhas.com/2014/05/an-introduction-to-c-templates/

C++ Template Functions

- Define with template<typename T> where
  - typename is replaced with a defined type (int, double, etc)
  - T is a placeholder to represent type in template
- Call with funcname<type>(params)
- <> brackets contain a list of template parameters



https://blog.feabhas.com/2014/05/an-introduction-to-c-templates/

C++ Template Functions

```
template <typename T>
T min (T a, T b)
{
  return a < b? a: b;
}

int main()
{
  int x;
  double y;

  x = min(10, 20);            // min<int> via compiler
  y = min(10.2, 20.9);        // min<double>

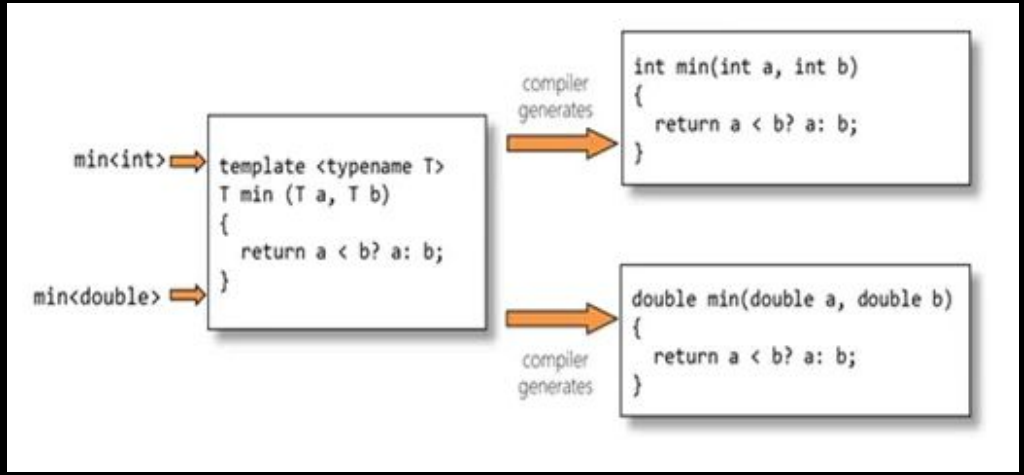
  x = min(10, 20.9);          // COMPILER ERROR
  x = min<int>(10, 20.9);     // okay - deduction not required
}
```

Type conversion from double to int

- In some cases the compiler can automatically determine the type, not necessary to specify

https://blog.feabhas.com/2014/05/an-introduction-to-c-templates/

C++ Compiler Steps



- Compiler generates code for you based on template argument
  - Support type checking by default
  - Do parameter evaluation before invoking the function (as a typical C function would)

https://blog.feabhas.com/2014/05/an-introduction-to-c-templates/

C++ Class Templates

```
1 template <class T>
2 class mypair {
3     T values [2];
4   public:
5     mypair (T first, T second)
6     {
7       values[0]=first; values[1]=second;
8     }
9 };
```

```
mypair<int> myobject (115, 36);
```

```
mypair<double> myfloats (3.0, 2.18);
```

- Similar to function templates
  - template <class T> to define
  - classname<typedef> to declare
- Define custom versions of a class based on a template type

http://www.cplusplus.com/doc/oldtutorial/templates/

C++ Standard Template Library (STL)

- Set of template based classes supported by C++ compilers for use in C++ programs
- string - dynamic sized char *
- vector, list, slist (dynamic arrays, double linked lists, single linked lists)
- map - Associative array
  - Array with possibly non numeric or non sequential index like device[“SerialNumber12345”]=”found”

http://www.cplusplus.com/doc/oldtutorial/templates/

C++ Streams

- C++ classes use the stream insertion operator << or >> to support data input/output
- C++ defines cin, cout, cerr and standard input,output and error streams

```
#include <iostream>
using namespace std;
```

```
1 cout << "Output sentence"; // prints Output sentence on screen
2 cout << 120;               // prints number 120 on screen
3 cout << x;                 // prints the value of x on screen
```

http://www.cplusplus.com/doc/tutorial/basic_io/

STL String

```cpp
#include <iostream>

int main(){
        std::string name;
        name = "Hall";
        std::cout << name << std::endl;
        return 0;
}
```

- Supports dynamically allocating, adding, appending, searching strings

```cpp
#include <iostream>

using namespace std;

int main(){
        string st1("Hello");
        string st2("World");
        cout << st1 + st2 << endl;
        st1 += "cpp";
        cout << st1 << endl;
        return 0;
}
```

```cpp
#include <iostream>
#include <string>

int main()
{
        std::string str1 ("I love C++");
        std::string str2 ("C++");
        std::cout << str1.find(str2) << std::endl;
        return 0;
}
```

Output

```
7
```

https://www.codesdope.com/cpp-stdstring/

STL Vector

```
#include <vector>
#include <iostream>

int main()
{
    std::vector<double> a;
    std::vector<double>::const_iterator i;
    a.push_back(1);
    a.push_back(2);
    a.push_back(3);
    a.push_back(4);
    a.push_back(5);
    for(i=a.begin(); i!=a.end(); ++i){
        std::cout<<(*i)<<std::endl;
    }
    return 0;
}
```

- Uses iterators
  - use classinstance.begin() and classinstance.end() to reference locations within the array
- Use push_back to add to the array dynamically
- Where is size specified?
  - Not specified, grows automatically

https://www.codeguru.com/cpp/cpp/cpp_mfc/stl/article.php/c4027/C-Tutorial-A-Beginners-Guide-to-stdvector-Part-1.htm
https://en.cppreference.com/w/cpp/container/vector

boost

- Peer reviewed portable C++ libraries which work with the STL
  - Suitable for eventual standardization
  - Permissively licensed
- Several useful libraries:
  - Thread - portable thread implementation across Windows/Mac/Linux
  - Regex - Regular Expression parser for text searching
  - Interprocess communication - Shared memory, semaphores/mutex, message queues

https://www.boost.org/
https://www.boost.org/doc/libs/1_71_0/more/getting_started/unix-variants.html